DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS
College of Arts and Sciences | University of the Philippines Visayas
Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020
Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Pitch Perfect

CMSC 197 GDD Pitching Guidelines

Prepared by: Rene Andre Bedonia Jocsing

Published: February 03, 2026

Every milestone in your portfolio project requires a pitch or presentation to the class. Whether it's your Week 2 concept, your Week 12 midterm demo, or your final Week 16 showcase, the fundamentals of effective communication remain constant. This guide provides **generic principles** applicable across all pitches.

Philosophy: Pitching is Technical Communication

A pitch is not a sales presentation. You are not selling a product to investors. You are **communicating technical and design decisions** to peers who understand game development. Your audience consists of fellow developers who will:

- Ask probing questions about your architecture
- Identify potential issues you may have overlooked
- Provide feedback based on their own implementation experiences
- Learn from your successes and failures

The goal is **peer review**, not persuasion. Be honest, be specific, be technical.

Universal Pitch Principles

These principles apply to **every** milestone presentation, regardless of development stage:

1. Respect the Time Limit

- **Week 12 & 16 (Milestones):** 20 minutes + Q&A
- **Other Weeks (Sub-Milestones):** 10 minutes + Q&A
- Practice beforehand. Time yourself. Cut content if needed.
- Going over wastes everyone's time and demonstrates poor planning

2. Lead with the Hook

Open with the **one sentence** that captures your game's essence:

- BAD: "We're making a game with some mechanics"
- GOOD: "Roguelike dungeon crawler with Pokemon-style type effectiveness"
- EXCELLENT: "Dead Cells meets Pokemon: real-time combat with strategic type matchups"

This anchors the audience's understanding for everything that follows.

3. Show, Don't Just Tell

Wherever possible, demonstrate rather than describe:

- **Week 2:** Mockups, reference images, mood boards, gameplay footage from reference games

- **Week 5+:** Live gameplay footage or direct demo
- **Architecture:** Diagrams showing scene hierarchy, signal flow, state transitions
- **Systems:** Code snippets illustrating key patterns (State, Command, Observer)

A 10-second gameplay clip communicates more than 2 minutes of verbal description.

4. Acknowledge Trade-offs and Constraints

Professional developers don't pretend perfection. They understand **engineering is about trade-offs**:

- "We chose A over B because of time constraints"
- "This is technical debt we'll address in Week X"
- "We cut feature Y to ensure core loop Z is polished"
- "Performance is suboptimal here; we're investigating optimization strategies"

This demonstrates maturity and self-awareness. Your peers will respect honesty over bravado.

5. Explain the "Why," Not Just the "What"

Don't just list features or systems. Justify your decisions:

- WEAK: "We have a health system"
- STRONG: "We use regenerating shields instead of health pickups because it maintains combat flow and reduces level design complexity"
- WEAK: "We implemented a State pattern"
- STRONG: "We used a State pattern for enemy AI because it cleanly separates behaviors and makes debugging individual states tractable"

The rationale behind decisions is as important as the decisions themselves.

6. Prepare for Q&A

The question period is not adversarial—it's collaborative debugging:

- Anticipate technical questions about your architecture
- If you don't know an answer, say so: "Good question, we haven't considered that yet"
- Take notes on feedback—this is free consulting from experienced peers
- Don't get defensive. If someone identifies a flaw, thank them.

Partner Coordination

Both team members should speak roughly equally during presentations. Divide sections beforehand:

- Partner A: Concept & core loop
- Partner B: Systems & architecture
- Both: Demo and Q&A

Uneven participation suggests uneven contribution, which affects your Individual Contribution grade.

Milestone-Specific Emphasis

While the above principles are universal, each milestone has distinct focus areas:

Week 2: Concept Pitch

Primary Goal: Establish feasibility and vision

Key Questions to Answer:

- What is the core gameplay loop in 3-5 bullet points?
- What are your genre inspirations?
- Why is this scope achievable in one semester?
- What are the biggest technical risks?

What NOT to Do:

- Vague genre labels (“It’s an RPG”)
- Feature lists without prioritization
- Ignoring technical feasibility

Week 5: Prototype Demo

Primary Goal: Prove the core loop is fun

Key Questions to Answer:

- Does the prototype demonstrate the hook?
- What did you learn that changes your Week 2 plan?
- What controls or mechanics didn’t work as expected?

What NOT to Do:

- Apologize for programmer art (it’s expected)
- Demo features outside the core loop
- Ignore prototype feedback in your revised scope

Week 8: Alpha Check-In

Primary Goal: Demonstrate architectural soundness

Key Questions to Answer:

- Are all major systems integrated?
- What design patterns are you using and why?
- What technical debt exists and when will you address it?

What NOT to Do:

- Claim “no bugs” (there are always bugs)
- Hide architectural issues
- Present a build that crashes during demo

Week 12: Midterm Presentation

Primary Goal: Showcase systems depth and iteration

Key Questions to Answer:

- How do your major systems interact (AI, combat, progression)?
- What did playtesting reveal?
- What’s the roadmap to beta?

What NOT to Do:

- Skip the demo to talk more
- Ignore playtester feedback
- Be vague about remaining work

Week 15: Beta Check-In

Primary Goal: Confirm feature lock and polish plan

Key Questions to Answer:

- What features are cut and why?
- What bugs won’t be fixed by Week 16?
- What’s your final week polish strategy?

What NOT to Do:

- Plan major new features
- Ignore known critical bugs
- Underestimate polish time

Week 16: Final Presentation

Primary Goal: Showcase a complete, polished game with reflection

Key Questions to Answer:

- What is the complete gameplay experience?
- What went well? What would you change?
- What were your key technical learnings?

What NOT to Do:

- Make excuses for missing features
- Demo bugs you could have fixed
- Skip the postmortem reflection

Visual Aid Guidelines

Slides are optional but often helpful. If you use them:

- **Minimal text:** Bullet points, not paragraphs
- **High-quality visuals:** Screenshots, diagrams, code snippets
- **Consistent formatting:** Don't mix 10 different fonts
- **Dark backgrounds:** Easier on eyes in classroom lighting
- **Readable font size:** 24pt minimum for body text

More slides does not equate to a better presentation. Respect your audience's time and attention.

Common Pitfalls

Avoid:

- Reading slides verbatim (your audience can read)
- Walls of text
- Low-resolution images or illegible diagrams
- Apologizing for slide quality (make better slides)
- Fancy transitions that waste time

Instead:

- Use slides as visual anchors for your speech
- Clear, high-contrast diagrams
- Code snippets with syntax highlighting
- Screenshots showing actual gameplay

Demonstration Best Practices

When showing your build (Week 5 onwards):

Technical Prep

- Test your build on the presentation machine **before** class
- Have a backup video recording in case of catastrophic failure
- Close unnecessary applications to prevent performance issues
- Use a wired mouse/keyboard if wireless input is flaky

Showing Gameplay

- Narrate what you're doing: "Now I'm using the dash ability to dodge"
- Show the core loop first, then edge cases or advanced features
- If a bug occurs, acknowledge it and move on (don't restart repeatedly)
- Keep demos under 5 minutes unless specifically showing a full playthrough

Known Issues

- Mention game-breaking bugs **before** you trigger them
- Explain workarounds: “Don’t press E near the door or it crashes”
- This shows you understand your codebase

Receiving Feedback

Your classmates’ questions and critiques are **valuable**. Treat them as free QA testing:

- **Listen actively:** Don’t interrupt or get defensive
- **Clarify if needed:** “Are you asking about the combat system or the leveling system?”
- **Take notes:** You won’t remember everything said
- **Thank contributors:** “Great point, we’ll investigate that”

If someone identifies a critical flaw you hadn’t considered, **that’s a win**. Better to find it in Week 8 than Week 16.

Giving Feedback to Others

When you’re in the audience:

- Ask **constructive** questions, not gotchas
- Focus on technical and design decisions, not art quality
- Suggest solutions if you see a problem: “Have you considered using a spatial hash for collision?”
- Be respectful. Everyone is learning.
- Remember that being quiet and failing to engage or ask questions means forfeiting your attendance for the session.

Good feedback helps your peers **and** prepares you to address similar issues in your own project.

Peer Learning Opportunity

Presentations are not just about showing your work—they’re about learning from others’ approaches:

- Which architecture patterns are they using effectively?
- What scope decisions seem to be working?
- Which technical challenges are common across projects?

Take notes during others’ presentations. You may solve a future problem using someone else’s solution.

Submission Requirements

For milestones with presentations (Weeks 12 and 16):

- **Updated living document** committed to repo before presentation
- **Build** exported and accessible (repo release)
- **Both partners present** unless excused absence with documentation

Missing any component results in incomplete grading for that milestone.

Final Thoughts

Pitching is a **skill** that improves with practice. Your Week 2 pitch will be rough. Your Week 16 presentation should be polished. Use each milestone as an opportunity to refine your communication:

- Clarity of vision
- Honesty about constraints
- Technical precision
- Receptiveness to feedback

These are not just academic skills. They are **professional competencies** you will use in every

software engineering role, whether in game development or beyond.

Take presentations seriously. Respect your audience's time. Communicate with clarity and humility. GLHF!

See the Portfolio Project document for grading rubrics.